

Version Control System, Git & GitHub

Interactive-style notes • Beginner → Interview Ready

Goal: Understand not only *what* Git commands do, but *why* they exist, what happens internally, and when to use them.

1. Version Control System (VCS) kya hota hai?

Simple definition: Version Control System ek system hai jo files/code ke changes ko time ke saath track karta hai. Aap dekh sakte ho kisne kya change kiya, kab kiya, purana version kya tha, aur zarurat padne par previous version par wapas ja sakte ho.

Real-life example

Socho aap ek project bana rahe ho. Monday ko **Project_v1**, Tuesday ko **Project_v2**, Wednesday ko **Project_final**, Thursday ko **Project_final_new**... Yeh manually manage karna dangerous hai. VCS isi problem ko systematically solve karta hai.

VCS kyun chahiye?

- History: changes ka complete record.
- Backup/recovery: previous working state par return kar sakte ho.
- Team collaboration: multiple developers same project par kaam kar sakte hain.
- Branching: feature ko main code se alag develop kar sakte ho.
- Comparison: do versions ke beech exact difference dekh sakte ho.
- Accountability: kis change ko kis author ne commit kiya, track hota hai.

Centralized vs Distributed VCS

Centralized: ek central server main repository rakhta hai; examples: SVN. **Distributed:** har developer ke paas repository ka complete history/local copy hota hai; Git iska popular example hai.

Exam point: Git = Distributed Version Control System (DVCS).

2. Git kya hai?

Git ek **distributed version control system** hai, originally software source code ke changes manage karne ke liye designed. Git aapke computer par locally repository aur history maintain karta hai.

Git vs GitHub — sabse important difference

Git: tool/software jo version control karta hai. **GitHub:** internet par Git repositories host/collaborate karne wali platform/service.

Analogy: Git = diary ka version/history system. GitHub = online locker + collaboration place jahan diary/project repository share kar sakte ho.

Basic architecture

Working Directory → Staging Area → Local Repository → Remote Repository (GitHub)

Flow:

Files edit → git add → git commit → git push → GitHub

GitHub → git pull/fetch → Local repository → Working directory

3. Repository kya hoti hai?

Repository (repo) ek project ka Git-managed area hai jisme files ke saath Git ki history/metadata bhi maintain hoti hai. Local repo aapke computer par hoti hai; remote repo GitHub jaise server par ho sakti hai.

git init

git init current folder ko Git repository banata hai. Isse hidden .git directory create hoti hai.

Example:

```
mkdir MyApp
cd MyApp
git init
```

Ab MyApp Git se track ki ja sakti hai.

4. Working Directory, Staging Area, Repository

Working Directory: jahan aap actual files edit karte ho.

Staging Area (Index): next commit me kaunse changes jayenge, uska selection area.

Repository: committed snapshots/history.

Why staging area?

Maan lo aapne 3 files change ki: Login.cs, Payment.cs, README.md. Aap sirf Login.cs ka change commit karna chahte ho. git add Login.cs se sirf wahi next commit ke liye stage hoga.

Flowchart: [EDIT FILE] → [WORKING DIRECTORY] → **git add** → [STAGING AREA] → **git commit** → [LOCAL REPOSITORY] → **git push** → [GITHUB]

5. git status

git status batata hai working tree me kya changed hai, kya staged hai, aur kya untracked hai.

Example: Aapne appsettings.json edit kiya. git status batayega ki file modified hai aur abhi staged nahi hai.

6. Commit kya hota hai? — Deep Explanation

Commit Git history me ek recorded checkpoint/snapshot hai. Isme project ki state aur metadata (author, time, message, parent relationship, etc.) associated hota hai.

Easy example

Aapne login feature complete kiya. Aap likhte ho:

```
git add .  
git commit -m "Add login API"
```

Iska matlab: current staged changes ko Git history me ek named checkpoint ke roop me record karo.

Commit message kyun?

Message future me samajhne ke liye hota hai ki commit ne kya change kiya. **Good:** "Add JWT authentication". **Bad:** "changes", "done", "final".

Commit ≠ Push

Commit: local Git repository me save/record.

Push: local commits ko remote repository (e.g. GitHub) par upload/send.

Aap 5 commits locally kar sakte ho aur baad me ek baar push kar sakte ho.

Commit ka conceptual graph

Initial commit (A) → Login commit (B) → Payment commit (C)
■ Feature branch commit (D)

7. git log

`git log` commit history dikhata hai. Useful when you want to know: kisne kya commit kiya, commit ID kya hai, kab hua, message kya tha.

8. git add

`git add file.txt` file ke current changes ko staging area me rakhta hai.

`git add .` current directory ke eligible changes ko stage kar sakta hai.

Important: `git add` ka matlab GitHub par upload karna nahi hai. Yeh sirf commit ke liye changes select karta hai.

9. git commit -m "..."

-m means commit message directly command me dena. Example:

```
git commit -m "Fix customer validation"
```

10. git diff

`git diff` generally unstaged changes ka difference dikhata hai. Staged changes dekhne ke liye `git diff --staged` useful hai.

11. .gitignore

.gitignore me aap specify karte ho ki kaunse files/folders Git ko track nahi karne chahiye.

Typical examples: build output, IDE folders, local secrets, environment-specific files. Example patterns:

```
bin/  
obj/  
.vs/
```

```
*.user  
.env
```

Important: .gitignore future untracked files ko ignore karta hai; agar secret pehle se commit ho chuka hai, sirf .gitignore add karna us history se automatically remove nahi karta.

12. Branch kya hoti hai?

Branch ek independent line of development hai. Isse aap feature/fix ko main development line se separate rakh sakte ho.

Example:

main: A → B → C

■ feature-login: D → E

Aap feature-login par kaam karte ho bina main branch ko directly disturb kiye.

Why branches?

- Feature development
- Bug fixes
- Experimentation
- Parallel team work
- Code review before merging

13. git branch

`git branch` branches list karta hai.

`git branch feature-login` new branch create karta hai.

14. git switch / git checkout

`git switch feature-login` branch change karne ka modern, focused command hai. Older workflows me `git checkout` bhi commonly used hai.

15. Merge kya hota hai?

Merge ek branch ke changes ko doosri branch ki history me integrate karta hai.

Example:

main: A → B → C

feature: ■ D → E

feature ko main me merge karne par feature ke changes main line me integrate hote hain.

Merge conflict

Jab Git automatically decide nahi kar pata ki same lines/related changes ko kaise combine karna hai, conflict hota hai. Developer ko file open karke correct version choose/modify karna hota hai, phir stage aur commit karna hota hai.

Typical conflict markers:

```
<<<<<<< HEAD
your changes
=====
incoming changes
>>>>>>> feature
```

16. Remote Repository

Remote repository ek Git repository hai jo aapke local machine ke bahar hosted hoti hai. GitHub par hosted repo commonly remote hoti hai.

git remote

`git remote -v` configured remote URLs dikhata hai.

`git remote add origin <repository-url>` local repo ko remote ke naam **origin** se connect karta hai.

origin koi magic keyword nahi; yeh conventionally remote ka default naam hota hai.

17. git push — Deep Explanation

`git push origin main` local main branch ke commits ko origin remote ki main branch par send karne ki request karta hai.

Flow: Local commit → network → remote repository → remote branch.

18. git pull

`git pull` generally remote changes ko fetch karta hai aur current branch me integrate karta hai. Conceptually: **fetch + integration**. Exact integration behavior configuration/history par depend kar sakta hai.

19. git fetch

`git fetch` remote se latest refs/commits laata hai bina current working branch me un changes ko automatically integrate kiye. Isliye review karne ke liye useful hai.

Pull vs Fetch

fetch: changes lao, integration khud decide karo.

pull: changes lao + current branch me integrate karne ki koshish karo.

20. clone

`git clone <url>` remote repository ki working copy/history ko local machine par laata hai.

Example: GitHub par team ka project hai → `git clone . . .` → project local machine par aa jata hai.

21. GitHub kya hai?

GitHub Git repositories ko host karne ke saath collaboration features deta hai: pull requests, code review, issues, permissions, Actions/automation, releases, etc.

GitHub Repository

Ek GitHub repo me source code, commits, branches, issues, pull requests aur other project metadata ho sakta hai.

22. Pull Request (PR)

Pull Request ek collaboration mechanism hai jisme aap propose karte ho ki ek branch ke changes ko target branch me integrate kiya jaye. Team reviewers code discuss/review kar sakte hain, checks run ho sakte hain, aur approval ke baad merge ho sakta hai.

Common flow:

Create branch → code → commit → push → Open PR → Review → Fix comments → Checks → Merge

23. Issue

GitHub Issue bug, task, feature request, question, etc. track karne ke liye use hota hai. Example: "Login API returns 500 for invalid token".

24. Complete Developer Workflow

Step 1: Clone project

```
git clone <url>
```

Step 2: New branch

```
git switch -c feature-login
```

Step 3: Code changes

Step 4: Check

```
git status
```

```
git diff
```

Step 5: Stage

```
git add .
```

Step 6: Commit

```
git commit -m "Add login API"
```

Step 7: Push

```
git push -u origin feature-login
```

Step 8: Create PR on GitHub

Step 9: Review + tests + merge

25. Reset vs Revert vs Restore

git restore working-tree/staged file state restore karne ke liye commonly used. **git reset** HEAD/branch pointer aur staging/working state ko different modes me move/change kar sakta hai; careless use history ko rewrite kar sakta hai. **git revert** existing commit ko reverse karne wala naya commit banata hai, isliye shared history me generally safer approach hoti hai.

Exam/interview memory: reset = history/pointer ko move/modify kar sakta hai; revert = inverse change ka new commit; restore = file content/staging restore.

26. HEAD kya hai?

HEAD Git ka symbolic reference hai jo usually currently checked-out branch/commit ko point karta hai. Simple language: "abhi main history ke kis point/branch par hoon?"

27. GitHub authentication

GitHub ke saath command-line operations ke liye authentication required ho sakta hai. Common modern approaches include HTTPS with credential helpers/tokens and SSH keys. Password-based Git operations are generally not the normal GitHub approach.

28. .git directory

.git repository ka internal metadata/database area hai. Isko delete karna local directory ko normal folder bana dega (files remain, Git history/configuration local repo ke roop me nahi rahegi).

29. HEAD, Working Tree, Index — mental model

Working Tree = aapke current files. Index/Staging = next commit ka selected content. HEAD = current committed point.
Yeh 3 concepts samajh liye to Git ka bahut bada part clear ho jata hai.

30. Interview Questions — Quick Revision

- Q1. Git kya hai?** Distributed version control system.
- Q2. GitHub kya hai?** Git repositories hosting/collaboration platform.
- Q3. Commit?** Git history me recorded checkpoint/snapshot.
- Q4. Push?** Local commits ko remote par send karna.
- Q5. Pull?** Remote changes fetch + current branch me integrate karna.
- Q6. Fetch?** Remote updates lana without automatically integrating into current branch.
- Q7. Branch?** Separate line of development.
- Q8. Merge?** Histories/changes ko integrate karna.
- Q9. PR?** Changes ko target branch me merge karne ka collaborative proposal/review workflow.
- Q10. .gitignore?** Certain untracked files/patterns ko tracking se exclude karna.

31. Command Cheat Sheet

Command	Meaning
git init	Current folder me Git repository initialize
git status	Current Git state/check changes
git add file	File changes stage
git add .	Eligible changes stage
git commit -m "msg"	Staged changes ko commit
git log	Commit history
git diff	Unstaged differences
git diff --staged	Staged differences
git branch	Branches list
git switch -c feature	Create + switch branch
git switch main	Switch branch
git merge feature	Feature changes merge
git clone URL	Remote repo local me clone
git remote -v	Remote URLs show
git fetch	Remote updates fetch
git pull	Fetch + integrate
git push origin main	Commits remote main par push
git restore file	File changes restore
git revert <commit>	Reverse change via new commit

32. Practice Exercise

Task: Create a local repo called GitPractice. Create README.md. Make first commit. Create feature-login branch. Add a login.txt file. Commit it. Push the branch to GitHub. Open a PR. Merge it.

Expected mental flow:

Folder → git init → working tree → git add → staging → git commit → local history → git branch → more commits → git push → GitHub → PR → review → merge

33. Golden Rules

1. Commit small, meaningful changes.
2. Write useful commit messages.

3. Pull/fetch and understand incoming changes before major work.
4. Never commit passwords/API secrets.
5. Use branches for isolated work.
6. Review `git diff` before committing important changes.
7. Understand whether a command changes local history or remote history before running it.

One-line memory trick: Edit → Add → Commit → Push → PR → Review → Merge

These notes are designed as a foundation. For production Git, the next topics to study are rebase, cherry-pick, stash, tags/releases, reflog, interactive rebase, conflict resolution, and GitHub Actions.